```
## ============================================================
## Document Similarity Lab
## TF-IDF Vectorisation + Cosine Similarity
## CSI3010 -- Data Mining & Warehousing
## ============================================================

## ---- 0. Packages ------------------------------------------
## install.packages(c("tm", "SnowballC", "ggplot2", "reshape2"))
library(tm)       # text preprocessing (Corpus, tm_map)
library(SnowballC)  # stemming
library(ggplot2)    # heatmap plot
library(reshape2)   # reshape similarity matrix for plotting

## ============================================================
## STEP 1: Text Preprocessing Pipeline
## ============================================================
## Lower-casing -> remove punctuation -> remove stop-words ->
## strip extra whitespace -> stemming

preprocess_text <- function(text) {
  corpus <- VCorpus(VectorSource(text))
  corpus <- tm_map(corpus, content_transformer(tolower))
  corpus <- tm_map(corpus, removePunctuation)
  corpus <- tm_map(corpus, removeWords, stopwords("english"))
  corpus <- tm_map(corpus, stripWhitespace)
  corpus <- tm_map(corpus, stemDocument)
  sapply(corpus, as.character)
}
```

```
## Worked example from the slides
example_sentence <- "Machine Learning is a subset of Artificial Intelligence."
cat("Original: ", example_sentence, "\n")
cat("Processed:", preprocess_text(example_sentence), "\n\n")

## ============================================================
## STEP 2: TF-IDF Vectorisation
## ============================================================

## ---- Sample corpus (D1-D4 related, D5 = unrelated control) ----
docs <- c(
  D1 = "Data science uses statistics and machine learning to extract knowledge from data.",
  D2 = "Machine learning is a subset of artificial intelligence that learns from data.",
  D3 = "Statistics is essential for data analysis and machine learning models.",
  D4 = "Deep learning is a branch of machine learning based on neural networks.",
  D5 = "Football is a popular sport played between two teams of eleven players."
)

## Preprocess every document with the Step-1 pipeline
docs_clean <- sapply(docs, preprocess_text)

## Build a Document-Term Matrix of raw counts (Bag of Words)
corpus <- VCorpus(VectorSource(docs_clean))
dtm_bow <- DocumentTermMatrix(corpus)
bow_matrix <- as.matrix(dtm_bow)
rownames(bow_matrix) <- names(docs)
```

```
cat("---- Bag-of-Words term-frequency matrix ----\n")
print(bow_matrix)

## ---- Manual TF-IDF: TF-IDF(t,d) = tf(t,d) * log(N / df(t)) ----
compute_tfidf <- function(bow) {
  N  <- nrow(bow)                    # total documents
  df <- colSums(bow > 0)             # document frequency per term
  idf <- log(N / df)                 # inverse document frequency
  tfidf <- sweep(bow, 2, idf, `*`)   # tf * idf, column-wise
  list(tfidf = tfidf, idf = idf)
}

tfidf_result <- compute_tfidf(bow_matrix)
tfidf_matrix <- tfidf_result$tfidf

cat("\n---- TF-IDF matrix ----\n")
print(round(tfidf_matrix, 3))

## ============================================================
## STEP 3: Cosine Similarity
## ============================================================
## cos(A,B) = (A . B) / (||A|| * ||B||)

cosine_similarity <- function(a, b) {
  sum(a * b) / (sqrt(sum(a^2)) * sqrt(sum(b^2)))
}
```

```
cosine_matrix <- function(mat) {
 n <- nrow(mat)
 sim <- matrix(0, n, n, dimnames = list(rownames(mat), rownames(mat)))
 for (i in 1:n) {
  for (j in 1:n) {
   sim[i, j] <- cosine_similarity(mat[i, ], mat[j, ])
  }
 }
 sim
}

sim_matrix <- cosine_matrix(tfidf_matrix)

cat("\n---- Full Cosine Similarity Matrix (TF-IDF) ----\n")
print(round(sim_matrix, 3))

## Most similar pair (excluding the diagonal)
sim_no_diag <- sim_matrix
diag(sim_no_diag) <- -1
best <- which(sim_no_diag == max(sim_no_diag), arr.ind = TRUE)[1, ]
cat(sprintf(
 "\nMost similar pair: %s <-> %s  (cosine = %.3f)\n",
 rownames(sim_matrix)[best[1]], colnames(sim_matrix)[best[2]],
 sim_matrix[best[1], best[2]]
))

## ---- Heat map (matches "Worked Example -- Result" slide) ----
sim_long <- melt(sim_matrix, varnames = c("Doc1", "Doc2"), value.name = "Similarity")
```

```
ggplot(sim_long, aes(x = Doc2, y = Doc1, fill = Similarity)) +
  geom_tile(color = "white") +
  geom_text(aes(label = sprintf("%.2f", Similarity)), size = 4) +
  scale_fill_gradient(low = "#FFFFE0", high = "#08306B", limits = c(0, 1)) +
  labs(title = "Cosine Similarity Heat Map (TF-IDF)", x = NULL, y = NULL) +
  theme_minimal(base_size = 12) +
  theme(panel.grid = element_blank())

## ============================================================
## More Worked Examples: Cosine Similarity in Other Scenarios
## ============================================================
## A lightweight TF-only cosine (no IDF) is often used for short,
## two-document comparisons like these.

tf_cosine <- function(text1, text2) {
  clean1 <- preprocess_text(text1)
  clean2 <- preprocess_text(text2)
  corpus2 <- VCorpus(VectorSource(c(clean1, clean2)))
  dtm2 <- as.matrix(DocumentTermMatrix(corpus2))
  cosine_similarity(dtm2[1, ], dtm2[2, ])
}
## Duplicate Question Detection
q1 <- "How can I improve my English speaking skills?"
q2 <- "What are some tips to improve spoken English?"
cat(sprintf("\nDuplicate Question Detection cosine (TF) = %.3f\n", tf_cosine(q1, q2)))
```

```
## Plagiarism / Paraphrase Detection

orig  <- "Climate change is primarily caused by human activities such as burning fossil fuels."

para  <- "Human activities, particularly the burning of fossil fuels, are the main cause of climate change."

cat(sprintf("Plagiarism/Paraphrase cosine (TF) = %.3f\n", tf_cosine(orig, para)))


## ============================================================

## Interpreting Cosine Similarity Scores

## ============================================================


interpret_score <- function(score) {

 if (score >= 0.80) "Near-duplicate"

 else if (score >= 0.50) "Strongly related"

 else if (score >= 0.20) "Loosely related"

 else "Unrelated"

}


cat("\n---- Interpretation of the D1-D5 similarity matrix ----\n")

for (i in 1:nrow(sim_matrix)) {

 for (j in i:ncol(sim_matrix)) {

  if (i != j) {

   cat(sprintf(

    "%s <-> %s : %.3f  (%s)\n",

    rownames(sim_matrix)[i], colnames(sim_matrix)[j],

    sim_matrix[i, j], interpret_score(sim_matrix[i, j])

   ))

  }

 }}
```